UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Binding
Casting in hierarchies
The hierarchy of classes in Java

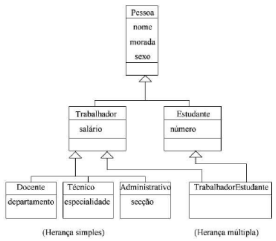
Readings: (

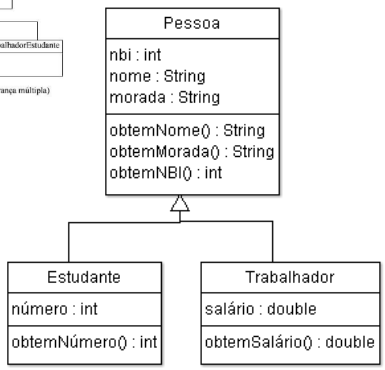
:chapt. 9)

1

Pessoa, Estudante e Trabalhador







From: https://www.freepik.com/free-vector/college-university-lectures-laboratories-practical-classes-library-study-places-breaks-outdoor-students-professors-cartoon-set-vector-illustration_26763297.htm

2

Remembering class Pessoa

```
class Pessoa {
    private final int nbi;
    private String nome, morada;
    public Pessoa(int bi,  String n, String m) {
            nbi = bi; nome = n;  morada = m;
    };
    public int obtemNbi() {return nbi;}
    public String obtemNome() {return nome;}
    public String obtemMorada () {return morada;}
    public String toString() {
            return new String ("Bi: " + nbi + “Nome: ” + nome + "Morada:
    " + morada);
    }
}
```

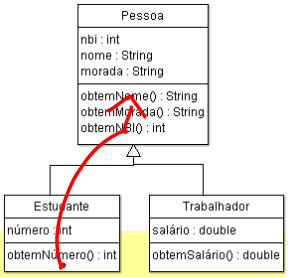
3

Remembering class Estudante

```
class Estudante extends Pessoa {
private final int  numero;
public Estudante(int bi, int no, String n, String m) {
            super(bi, n, m);
            numero = no;
}
public Estudante(int bi, int no, String n) {
    this(bi, no, n, “Desconhecida”);
}
public int obtemNumero() {return numero; }
public String toString() {
            String s = super.toString();
            String e = new String("Numero estudante: " + obtemNumero());
            return s+e;
}}
```

4

Check point: what's the output and why?




```
public class Main {
        public static void main(String[] args) {
          Queue<Pessoa> q = new LinkedList<>();
          Estudante e = new Estudante(222,123, "Joao", "Olhao");
          Pessoa p = new Pessoa(111,"Ana","Faro");
          q.add(e);
          q.add(p);
          while(!q.isEmpty())
                IO.println(q.remove().obtemNome());
        }
}
```

5

Check point: what's the output and why?

```
public class Main {
        public static void main(String[] args) {
          Queue<Pessoa> q = new LinkedList<>();
          Estudante e = new Estudante(222,123, "Joao", "Olhao");
          Pessoa p = new Pessoa(111,"Ana","Faro");
          q.add(e);
          q.add(p);
          while(!q.isEmpty())
                IO.println(q.remove());
        }
}
```

Calls method **toString()** ***dynamically***

```
C:\WINDOWS\system32\cmd.exe
\poo\teóricas\T6 - Modelos e mecanismos de herança\code>java Main
Bi: 222 Nome: Joao Morada: Olhao Numero estudante: 123
Bi: 111 Nome: Ana Morada: Faro
```

6

Binding

- Binding occurs whenever a correspondence is established between a name (e.g., a variable or method name) and its corresponding value.
- Binding occurs at various times in the life of a program (e.g., compile time, load time, run time).
- Kinds of binding:
  - static binding
  - dynamic binding
  - late binding

7

Binding

- Java has static binding for variables and dynamic binding for methods.

WARNING

The following example illustrates the difference between static and dynamic binding.

The employment of public instance variables should be avoided. Therefore, the following code should not be taken as a good programming practice.

8

Binding

```
class A {
   public String a="A.a";
   public String f() {return "A.f";}}
class B extends A {
   public String a ="B.a";
   public String f() {return "B.f";}
}


// …

A a = new A();
B b = new B();
System.out.println(a.f());
System.out.println(b.f());
System.out.println(a.a);
System.out.println(b.a);
A x = b;
System.out.println(x.f());
System.out.println(x.a);
```

```
// Console

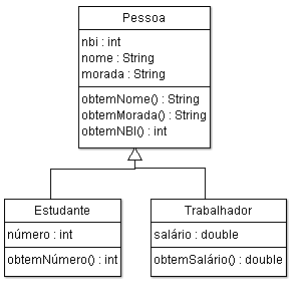
A.f
B.f
A.a
B.a

B.f
A.a
```

9

Casting in hierarchies

```
Pessoa p = new Pessoa(1, "Ana", "Faro");
Estudante e = new Estudante(2, 111, "Maria", "Lisboa" );
Trabalhador t = new Trabalhador(3, "Rui", "Estoi", 1.5f);
```




```
p=t; // OK (upcast)
System.out.println(p.obtemMorada());
```

```
e=p; // ERROR! (downcast)
System.out.println(e.obtemNumero());
```

```
t=e; // ERROR! (incompatible types)
System.out.println(t.obtemSalario());
```

10

Casting in hierarchies

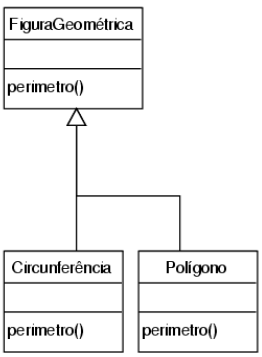
```
Pessoa p = new Pessoa(1, "Ana", "Faro");
Estudante e = new Estudante(2, 111, “Maria”, “Lisboa” );
Trabalhador t = new Trabalhador(3, “Rui", “Estoi", 1.5f);
```

```
p = t;
t = (Trabalhador) p;
System.out.println(t.obtemSalário())
```

```
If (p instanceof Trabalhador)
   t = (Trabalhador) p;
```

11

Check point



- Which conversions are possible, if any:

```
Circunferencia c;
FiguraGeometrica f;
Poligono p;

// …

c = f;    //    i)
f = p;    //    ii)
c=p;      //    iii)
p=c;      //    iv)
f=c;      //    v)
```

12

UAlg FCT
UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Hierarchies of classes

Readings: ( :chapt. 9)

13

Outline

- Casting is hierarchies
- The hierarchy of classes in Java.
- Abstract classes
- The class Object
- Rules for method redefinition
- Type-wrapper classes

14

Pois

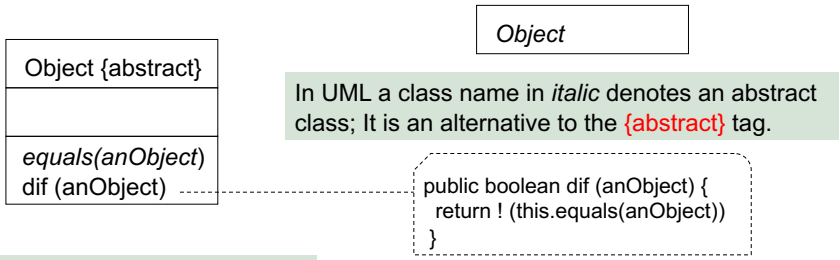
é

15

Object {abstract}

18

Exploiting inheritance (Java sintaxe but not Java)

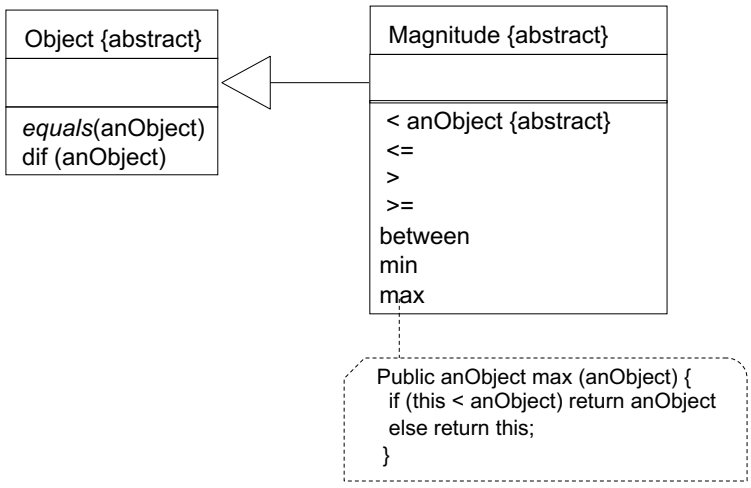


In UML a method signature in *italic* is an abstract method

(it has no body; it must be defined in a subclass)

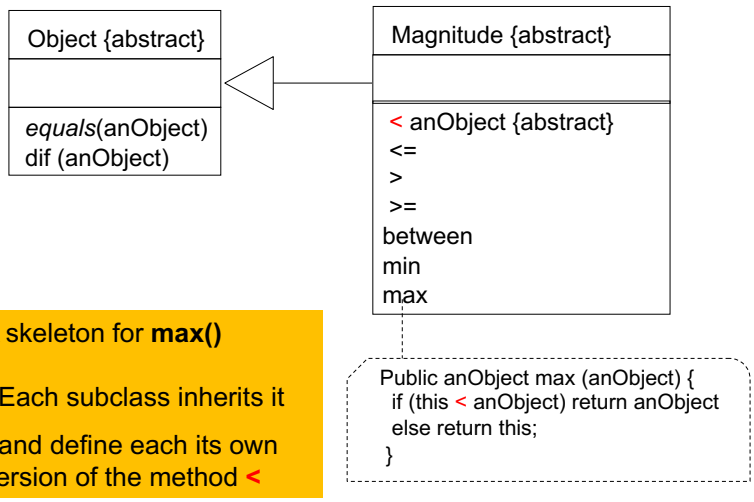
19

Exploiting inheritance (Java sintaxe but not Java)



20

Exploiting inheritance (Java sintaxe but not Java)

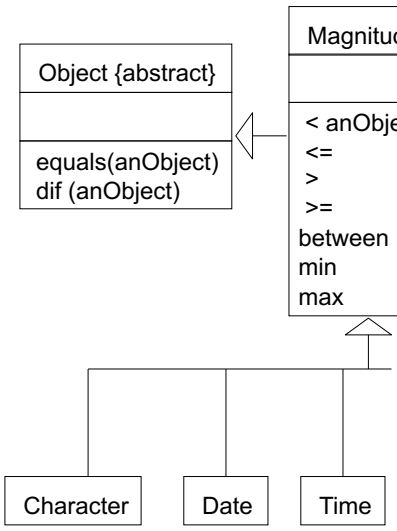


A skeleton for **max()**

- Each subclass inherits it
- and define each its own version of the method **<**

21

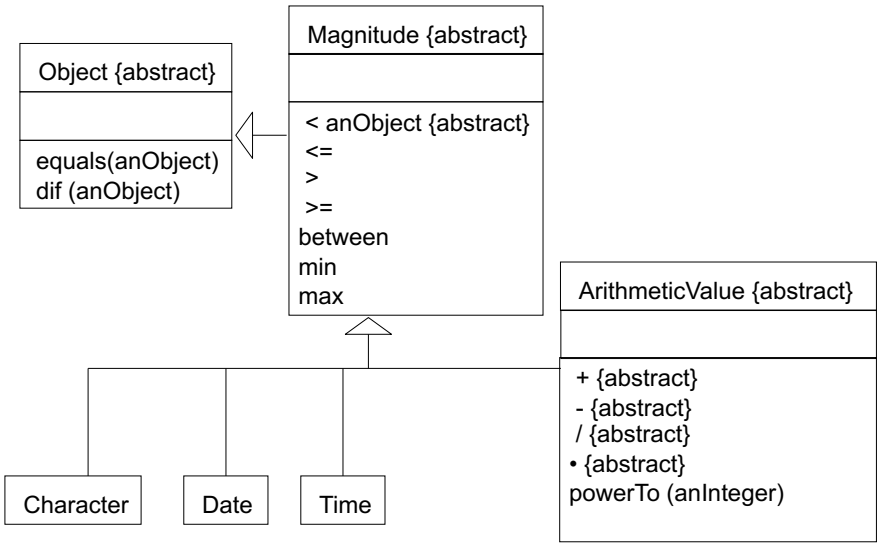
Exploiting inheritance



these subclasses **inherit** the method: **max** and **implement their own specific version** of method: **<**

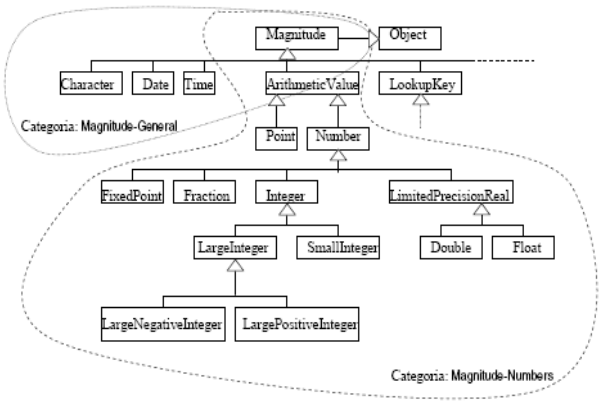
22

Exploiting inheritance



23

Subhierarchy Magnitude of Smalltalk



24

Abstract class

- An abstract class is a class that cannot have instances;
- Abstract classes are useful both to endow state and behaviour to their subclasses, and to specify a set of methods that subclasses are required to implement

25

The keyword *abstract* in methods

- A method can be declared ***abstract***. An abstract method does not have body
- A class with an ***abstract*** method must be declared abstract
- Abstract class methods cannot be declared final, nor private.

Why?

26

The keyword *final* in classes and methods

- A class declared ***final*** cannot have subclasses. All methods of a final class are final.
- A method declared ***final*** cannot be redefined. As a consequence, dynamic binding will not apply, and the compiler will be able to use inline code expansion to increase eficiency.

27